

实 验 报 告

课程名称：机器学习及优化

姓名：王志豪

学号：08232337

班级：2023 大数据一班

日期：2026. 4. 22

中国矿业大学计算机学院/人工智能学院

实验名称 决策树与随机森林分类 实验日期 2026. 4. 22

实验报告内容: 1. 实验目的 2. 实验内容 3. 实验步骤 4. 实验结果 5. 实验体会

一、实验目的

通过实验更好地掌握机器学习及优化的相关概念、技术、原理、应用等；通过实验提高学生编写实验报告、总结实验结果的能力；使学生对机器学习模型、算法等有比较深入的认识。需掌握的知识点如下：

- 掌握机器学习中涉及的相关概念、模型、算法
- 熟悉机器学习模型训练、验证、测试的流程
- 熟悉常用的数据预处理方法
- 掌握决策树、随机森林的表示、求解及编程

二、实验内容

基于 Adult 数据集，完成关于收入是否大于 50K 的贪心决策树分类、随机森林分类模型训练、测试与评估。

1) 准备数据集并认识数据

- 下载 Adult 数据集：<http://archive.ics.uci.edu/ml/datasets/Adult>
- 了解数据集各个维度特征及预测值的含义。

2) 探索数据并预处理数据

- 观察数据集各个维度特征及预测值的数值类型与分布。
- 预处理各维度特征，参考：<https://blog.csdn.net/SanyHo/article/details/105304292>

3) 训练模型

- 编程实现训练数据集上贪心决策树、随机森林的构建。

4) 测试和评估模型

- 在测试数据集上计算所训练模型的准确率、AUC 等指标。

三、实验步骤

实验原理：

(1) 决策树：

A. 决策树的本质

决策树是一种基于特征对样本进行分类的模型，可以看作是在特征空间上划分区域，每个区域对应一个预测类别。给定一个样本的特征取值，决策树通过一系列“是/否”判断（即内部节点的特征测试）将其引导至某个叶节点，叶节点存储的是该路径下最可能的类别标签。这种模型属于“基于计数”的方法——叶节点的类别由训练集中落入该区域的样本的多数类别决定。

B. 关键度量指标

- **熵 (Entropy):** 衡量随机变量不确定性的指标。熵值越大，变量的取值越混乱。对于离散随机变量 X ，其熵为

$$H(X) = - \sum_i p_i \log p_i$$

- **条件熵 (Conditional Entropy):** 在已知另一个随机变量 X 的条件下， Y 剩余的不确定性，记作 $H(Y | X)$ 。计算公式为

$$H(Y | X) = \sum_x p(x) H(Y | X = x)$$

- **信息增益 (Information Gain):** 使用某个特征 A 对数据集 D 进行划分后, 分类不确定性减少的程度。信息增益越大, 说明该特征对分类的贡献越大。其定义为

$$G(D, A) = H(D) - H(D | A)$$

其中 $H(D)$ 是类别标签的熵, $H(D | A)$ 是给定特征 A 条件下类别的条件熵。

C. 决策树的构建流程 (ID3 算法)

- **特征选择:** 在每一步, 从当前数据集中选择一个能带来最大信息增益的特征作为分裂依据。本实验采用 ID3 算法, 即信息增益准则。
- **树的生成:** 根据选定的特征将数据集划分成若干子集, 对每个子集递归执行上述特征选择与分裂, 直到子集中的样本全部属于同一类别, 或没有剩余特征可用。
- **剪枝 (可选):** 为了防止模型过度适应训练数据 (过拟合), 需要对生成的完整决策树进行简化, 移除一些对泛化能力贡献不大的分支。

(2) 随机森林

A. 随机森林的基本概念

随机森林是一种集成学习方法, 它通过构建多棵互不相同的决策树并将它们的预测结果进行投票 (或平均) 来做出最终决策。它可以处理分类和回归问题。本实验中的基决策树同样采用 ID3 算法。

B. 与单棵决策树的两大区别

随机森林在集成过程中引入了两种随机性, 使得各棵决策树彼此差异化:

- **数据采样随机性:** 对于每棵决策树, 采用**有放回**的随机抽样方式从原始数据集中抽取同样数量的样本 (允许重复)。这样, 每棵树的训练集都不同。
- **特征选择随机性:** 在每棵树的每个节点分裂时, 不是从全部特征中选择最佳分裂特征, 而是**随机选取一个子集** (子集大小 n 为超参数), 然后仅从这个子集中选出最优特征进行分裂。

C. 构建与预测步骤

- **生成子数据集:** 对原始数据集进行有放回抽样, 生成与原始数据集规模相同的子数据集 (Bootstrap 样本)。
- **随机选取特征子集:** 为每棵树的每个分裂节点随机抽取部分特征, 从中选择最佳分裂特征。
- **构建多棵决策树:** 用上述子数据集和特征子集策略, 递归生成多棵决策树 (不进行剪枝, 通常让树充分生长)。
- **集成预测:** 对于新样本, 让每一棵决策树分别输出一个预测类别, 最后通过**多数投票**决定随机森林的最终输出。

这种随机化设计增强了模型的多样性, 从而降低了过拟合风险, 提高了泛化能力。

1. 导入相应库支持

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from math import log
import operator
import random
from random import randrange
from collections import Counter
```

2. 导入数据集, 观察数据特点

训练集数据如下:

```

RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    32561 non-null  int64
1   workclass               32561 non-null  object
2   fnlwgt                  32561 non-null  int64
3   education               32561 non-null  object
4   education-num           32561 non-null  int64
5   marital-status          32561 non-null  object
6   occupation              32561 non-null  object
7   relationship            32561 non-null  object
8   race                    32561 non-null  object
9   sex                    32561 non-null  object
10  capital-gain             32561 non-null  int64
11  capital-loss             32561 non-null  int64
12  hours-per-week           32561 non-null  int64
13  native-country           32561 non-null  object
14  income                   32561 non-null  object
dtypes: int64(6), object(9)

```

测试集数据如下：

```

Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    16282 non-null  object
1   workclass               16281 non-null  object
2   fnlwgt                  16281 non-null  float64
3   education               16281 non-null  object
4   education-num           16281 non-null  float64
5   marital-status          16281 non-null  object
6   occupation              16281 non-null  object
7   relationship            16281 non-null  object
8   race                    16281 non-null  object
9   sex                    16281 non-null  object
10  capital-gain            16281 non-null  float64
11  capital-loss            16281 non-null  float64
12  hours-per-week           16281 non-null  float64
13  native-country           16281 non-null  object
14  income                   16281 non-null  object
dtypes: float64(5), object(10)

```

3. 进行数据预处理

(1). 处理缺失值

```

# 处理缺失值，将含有'?'的行去掉
for i in train_set.columns:
    test_set = test_set[test_set[i] != '?']
    train_set = train_set[train_set[i] != '?']

# 重新规划数据集的index
test_set.index = range(0, len(test_set))
train_set.index = range(0, len(train_set))

```

(2). 删除 fnlwgt 列（即序号列）该列不影响分类结果

```

# 删除fnlwgt列
train_set.drop('fnlwgt', axis=1, inplace=True)
test_set.drop('fnlwgt', axis=1, inplace=True)

```

(3). 删除 Education 列

```
# 删除Education列
train_set.drop(['education'], axis=1, inplace=True)
test_set.drop(['education'], axis=1, inplace=True)
```

(4). 将一些连续值处理为离散值，合并一些 object 类型的属性列值。观察各属性列值的特点后，做出以下处理

A. 将 age 属性列值范围划分为 0-25,25-50,50-75,75-100

```
# 1. 将age属性划分为0-25 25-50 50-75 75-100
age = np.copy(train_set['age'])
age_test = np.copy(test_set['age'])
def transform1(age, size):
    for i in range(size):
        if age[i] >= 0 and age[i] < 25:
            age[i] = 0
        elif age[i] >= 25 and age[i] < 50:
            age[i] = 1
        elif age[i] >= 50 and age[i] < 75:
            age[i] = 2
        elif age[i] >= 75:
            age[i] = 3
    return age
age = transform1(age, m)
age_test = transform1(age_test, n)
train_set['age'] = age
test_set['age'] = age_test
```

```
Traceback (most recent call last):
  File "f:\课设\机器学习及优化\作业四\code.py", line 57, in <module>
    age_test = transform1(age_test, n)
  File "f:\课设\机器学习及优化\作业四\code.py", line 47, in transform1
    if age[i] >= 0 and age[i] < 25:
TypeError: '>=' not supported between instances of 'str' and 'int'
```

在真实跑程序的过程中，参考程序会出现一处错误：在读入数据后，数据默认是字符串类型，而直接对字符串类型进行数值比较会报错，因此需要加以改进，改进思路是，把数据读取和清洗前移，去掉字符串空白，把 ? 统一当成缺失值，再把数值列强制转成数字，这样问题就解决了。

B. 处理 capital-gain 属性列，>0 的值用 1 代换，=0 的值用 0 代换

```
# 2. 将capital-gain属性 >0的值用1替换，=0的值用0替换
gain = np.copy(train_set['capital-gain'])
gain_test = np.copy(test_set['capital-gain'])
def transform2(gain, size):
    for i in range(size):
        if gain[i] == 0:
            gain[i] = 0
        elif gain[i] > 0:
            gain[i] = 1
    return gain
gain = transform2(gain, m)
gain_test = transform2(gain_test, n)
train_set['capital-gain'] = gain
test_set['capital-gain'] = gain_test
```

C. 将 captional-loss 属性划分为>0 =0 两类

```
# 3. 将capital-loss属性划分为>0, =0两类
loss = np.copy(train_set['capital-loss'])
loss_test = np.copy(test_set['capital-loss'])
loss = transform2(loss, m)
loss_test = transform2(loss_test, n)
train_set['capital-loss'] = loss
test_set['capital-loss'] = loss_test
```

D. 将 hours-per-week 划分为 40 等 3 类

```
# 4. 将hours-per-week划分为 <40, ==40, >40
hours = np.copy(train_set['hours-per-week'])
hours_test = np.copy(test_set['hours-per-week'])
def transform3(hours, size):
    for i in range(size):
        if hours[i] < 40:
            hours[i] = 0
        elif hours[i] == 40:
            hours[i] = 1
        elif hours[i] > 40:
            hours[i] = 2
    return hours
hours = transform3(hours, m)
hours_test = transform3(hours_test, n)
train_set['hours-per-week'] = hours
test_set['hours-per-week'] = hours_test
```

E. 将 Country 划分为 USA、not USA。

```
# 5. 将Country划分为USA, not USA两类
cty = np.copy(train_set['native-country'])
cty_test = np.copy(test_set['native-country'])
def transform4(cty, size):
    for i in range(size):
        if cty[i] == "United-States":
            cty[i] = 0
        elif cty[i] != "United-States":
            cty[i] = 1
    return cty
cty = transform4(cty, m)
cty_test = transform4(cty_test, n)
train_set['native-country'] = cty
test_set['native-country'] = cty_test
```

F. 将 workclass 分为 Freelance、other、Proprietor、Private、Government 五类，其中 Freelance other Private 是本身就有的类，不做处理

```
# 6. 将workclass分为Freelance, other, Proprietor, Private, Government五类
workclass = np.copy(train_set['workclass'])
workclass_test = np.copy(test_set['workclass'])
def transform6(workclass, size):
    for i in range(size):
        if workclass[i] == "Federal-gov" or workclass[i] == "Local-gov" or workclass[i] == "State-gov":
            workclass[i] = "Government"
        elif workclass[i] == "Self-emp-not-inc" or workclass[i] == "Self-emp-inc":
            workclass[i] = "Proprietor"
    return workclass
workclass = transform6(workclass, m)
workclass_test = transform6(workclass_test, n)
train_set['workclass'] = workclass
test_set['workclass'] = workclass_test
```

G. 将 education-num 分为 0-5:0、5-10:1、>=10:2

```
# 7. 将education-num分为0-5:0, 5-10:1, >=10:2
edu = np.copy(train_set['education-num'])
edu_test = np.copy(test_set['education-num'])
def transform5(edu, size):
    for i in range(size):
        if edu[i] < 5:
            edu[i] = 0
        elif edu[i] >= 5 and edu[i] < 10:
            edu[i] = 1
        elif edu[i] >= 10:
            edu[i] = 2
    return edu
edu = transform5(edu, m)
edu_test = transform5(edu_test, n)
train_set['education-num'] = edu
test_set['education-num'] = edu_test
```

H. 将 marital_status 分为两类: married 和 not-married

```
# 8. 将marital_status分为两类: married, not-married
mari = np.copy(train_set['marital-status'])
mari_test = np.copy(test_set['marital-status'])
def transform8(mari, size):
    for i in range(size):
        if mari[i] == "Divorced" or mari[i] == "Never-married" or mari[i] == "Separated" or mari[i] == "Widowed":
            mari[i] = "not-married"
        else:
            mari[i] = "married"
    return mari
mari = transform8(mari, m)
mari_test = transform8(mari_test, n)
train_set['marital-status'] = mari
test_set['marital-status'] = mari_test
```

I. 将 occupation 分为 High、Med、Low 三类

```
# 9. 将occupation分为High, Med, Low三类
occu = np.copy(train_set['occupation'])
occu_test = np.copy(test_set['occupation'])
def transform9(occu, size):
    for i in range(size):
        if occu[i] == "Prof-specialty" or occu[i] == "Exec-managerial":
            occu[i] = "High"
        elif occu[i] == "Tech-support" or occu[i] == "Transport-moving" or \
            occu[i] == "Protective-serv" or occu[i] == "Sales" or occu[i] == "Craft-repair" \
            or occu[i] == "Armed-Forces":
            occu[i] = "Med"
        else:
            occu[i] = "Low"
    return occu
occu = transform9(occu, m)
occu_test = transform9(occu_test, n)
train_set['occupation'] = occu
test_set['occupation'] = occu_test
```

J. 将 relationship 分为 Husband、Other、Wife

```
# 10. 将relationship分为Husband, Other, Wife
reship = np.copy(train_set['relationship'])
reship_test = np.copy(test_set['relationship'])
def transform7(reship, size):
    for i in range(size):
        if reship[i] != "Husband" and reship[i] != "Wife":
            reship[i] = "Other"
    return reship
reship = transform7(reship, m)
reship_test = transform7(reship_test, n)
train_set['relationship'] = reship
test_set['relationship'] = reship_test
```

K. 将 race 分为两类: White 和 Other

```
# 11. 将race分为两类: White和Other
race = np.copy(train_set['race'])
race_test = np.copy(test_set['race'])
def transform10(race, size):
    for i in range(size):
        if race[i] != "White":
            race[i] = "Other"
    return race
race = transform10(race, m)
race_test = transform10(race_test, n)
train_set['race'] = race
test_set['race'] = race_test
```

(5). 处理标签列及测试集

```
# 对income标签列进行标签编码
label_encoder = LabelEncoder()
train_set['income'] = label_encoder.fit_transform(train_set['income'])
test_set['income'] = label_encoder.fit_transform(test_set['income'])

# 将测试集转为列表
y_test = test_set['income'].values.tolist()
y_train = train_set['income'].values.tolist()
test_set = test_set.values.tolist()
```

ID3 决策树

1. 创建数据集函数

```
# 创建数据集函数
def createDataSet():
    dataSet = train_set.values.tolist()
    featureName = ['age', 'workclass', 'education-num', 'marital-status', 'occupation', 'relationship', 'race', 'sex',
                   'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']
    return dataSet, featureName
```

```
# 构建决策树并评估
dataSet, featureName = createDataSet()
```


2. 创建决策树

```
# 创建决策树
def createTree(dataSet, featureName, chooseBestFeatureToSplitFunc=chooseBestFeatureToSplitByID3):
    classList = [example[-1] for example in dataSet]
    if classList.count(classList[0]) == len(classList):
        return classList[0]
    if len(dataSet[0]) == 1:
        return majorityCnt(classList)
    bestFeat = chooseBestFeatureToSplitFunc(dataSet)
    bestFeatLabel = featureName[bestFeat]
    myTree = {bestFeatLabel: {}}
    del(featureName[bestFeat])
    featValues = [example[bestFeat] for example in dataSet]
    uniqueVals = set(featValues)
    for value in uniqueVals:
        subLabels = featureName[:]
        myTree[bestFeatLabel][value] = createTree(splitDataSet(dataSet, bestFeat, value), subLabels)
    return myTree
```

```
myTree = createTree(dataSet, featureName)
```

- (1). 用 classList 列表来装数据集上的分类列

```
classList = [example[-1] for example in dataSet]
```

- (2). 判断 classList 列表中装的值是否都相同，如果相同（类别完全相同）则返回该相同值

```
if classList.count(classList[0]) == len(classList):
    return classList[0]
```

- (3). 当只有一个特征的时候，遍历所有实例返回出现次数最多的类别（调用函数 majorityCnt）

```
if len(dataSet[0]) == 1:
    return majorityCnt(classList)
```

- (4). 找出决策树最佳划分特征（调用函数 chooseBestFeatureToSplitFunc），构建字典

```
bestFeat = chooseBestFeatureToSplitFunc(dataSet)
bestFeatLabel = featureName[bestFeat]
myTree = {bestFeatLabel: {}}
del(featureName[bestFeat])
```

其中 chooseBestFeatureToSplitByID3 函数又调用了以下函数来计算每个特征的信息增益来比较选出最佳特征：

A. 计算信息熵

```
# 计算信息熵
def calcShannonEnt(dataSet):
    numEntries = len(dataSet)
    labelCounts = {}
    for featVec in dataSet:
        currentLabel = featVec[-1]
        if currentLabel not in labelCounts.keys():
            labelCounts[currentLabel] = 0
        labelCounts[currentLabel] += 1
    shannonEnt = 0.0
    for key in labelCounts:
        prob = float(labelCounts[key]) / numEntries
        shannonEnt -= prob * log(prob, 2)
    return shannonEnt
```

B. 计算条件熵

```
# 计算条件熵
def calcConditionalEntropy(dataSet, i, featList, uniqueVals):
    ce = 0.0
    for value in uniqueVals:
        subDataSet = splitDataSet(dataSet, i, value)
        prob = len(subDataSet) / float(len(dataSet))
        ce += prob * calcShannonEnt(subDataSet)
    return ce
```

C. 计算信息增益

```
# 计算信息增益
def calcInformationGain(dataSet, baseEntropy, i):
    featList = [example[i] for example in dataSet]
    uniqueVals = set(featList)
    newEntropy = calcConditionalEntropy(dataSet, i, featList, uniqueVals)
    infoGain = baseEntropy - newEntropy
    return infoGain
```

- (5). 用 featValues 列表装该最佳特征列的所有值，uniqueVals 装该列中出现过的值（只记值的种类）

```
featValues = [example[bestFeat] for example in dataSet]
uniqueVals = set(featValues)
```

- (6). 遍历 uniqueVals，当按照 uniqueVals 中的值 value 划分数据集（调用函数 splitDataSet）之后，再在新的数据集重新寻找最佳特征，递归创建决策树的分支

```
for value in uniqueVals:
    subLabels = featureName[:]
    myTree[bestFeatLabel][value] = createTree(splitDataSet(dataSet, bestFeat, value), subLabels)
```

```
# 分割数据集
def splitDataSet(dataSet, axis, value):
    retDataSet = []
    for featVec in dataSet:
        if featVec[axis] == value:
            reduceFeatVec = featVec[:axis]
            reduceFeatVec.extend(featVec[axis+1:])
            retDataSet.append(reduceFeatVec)
    return retDataSet
```

决策树中分支路停止创建的条件为：

新的数据集上的类别列的值都相同时，或者新的数据集上只剩下分类列这一列（无特征列时）。

```

if classList.count(classList[0]) == len(classList):
    return classList[0]
if len(dataSet[0]) == 1:
    return majorityCnt(classList)

```

递归完后决策树创建成功。

3. 根据决策树对测试集上的数据进行分类

```

featLabels = ['age', 'workclass', 'education-num', 'marital-status', 'occupation', 'relationship', 'race', 'sex',
              'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']
featLabels_train = ['age', 'workclass', 'education-num', 'marital-status', 'occupation', 'relationship', 'race', 'sex',
                    'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']

classList = []
classList_train = []

for i in range(len(test_set)):
    classLabel = classify(myTree, featLabels, test_set[i])
    classList.append(classLabel)

```

分类函数 `classify` 定义如下：

```

# 分类函数
def classify(inputTree, featLabels, testVec):
    firstStr = list(inputTree.keys())[0]
    secondDict = inputTree[firstStr]
    featIndex = featLabels.index(firstStr)
    key = testVec[featIndex]
    if key not in secondDict:
        key = list(secondDict.keys())[0]
    valueOfFeat = secondDict[key]
    if isinstance(valueOfFeat, dict):
        classLabel = classify(valueOfFeat, featLabels, testVec)
    else:
        classLabel = valueOfFeat
    return classLabel

```

由于决策树完全基于训练集构建，其分支结构只覆盖了训练数据中出现过的特征取值组合。当测试样本的某个特征取值在训练集中从未出现时，决策树沿着对应路径会找不到匹配的子节点，从而无法分类。例如，训练集中属性 A 取值为 a 时，属性 B 只出现过 b, c, d 三种取值，而测试样本中 $A = a$ 且 $B = f$ ，这种情况在训练过程中没有见过，树中不存在 $B = f$ 的分支。

理想的处理方式是：回退到属性 $A = a$ 的节点，统计该节点下训练样本的类别分布，将测试样本判为出现频率最高的类别。但由于程序已经编写完成，修改逻辑较为繁琐，本实验采用一种简化策略：直接将这种未定义情况归类为属性 $A = a$ 且属性 $B = b$ （即第一个存在的分支）所对应的类别。

4. 计算准确率和 AUC 值

```
# 计算准确率
correct_num = 0
for i in range(n):
    if classList[i] == y_test[i]:
        correct_num += 1
rate = correct_num / n
print('决策树准确率:', rate)

correct_num_train = 0
for i in range(m):
    if classList_train[i] == y_train[i]:
        correct_num_train += 1
rate_train = correct_num_train / m
print('决策树训练集上准确率:', rate_train)

from sklearn.metrics import roc_auc_score
auc_score = roc_auc_score(y_test, classList)
print('决策树AUC值:', auc_score)
```

随机森林

1. 定义参数

```
# 随机森林
n_features = 8
n_trees = 10
```

2. 根据样本集及 n_trees 参数构建随机森林

```
TreeList = []
def RandomForest(dataSet, n_trees):
    for i in range(n_trees):
        final_subData, subFeature = generateDataSet(dataSet, featLabels, n_features)
        myTree = createTree(final_subData, subFeature)
        TreeList.append(myTree)
    return TreeList

RandomForest(dataSet, n_trees)
```

每一个子决策树的构建步骤如下:

(1). 从样本集中随机构造出的数据子集

```
def get_subsample(dataSet):
    subdataSet = []
    lenSubdata = len(dataSet)
    while len(subdataSet) < lenSubdata:
        index = randrange(len(dataSet) - 1)
        subdataSet.append(dataSet[index])
    return subdataSet
```

(2). 随机选中 n_features 个特征

```
def get_subfeature(featLabels, n_features):
    subFeature = []
    subFeatIndex = random.sample(range(0, len(featLabels) - 1), n_features)
    for i in range(len(subFeatIndex)):
        subFeature.append(featLabels[subFeatIndex[i]])
    subFeature.append('income')
    subFeatIndex.append(12)
    return subFeature, subFeatIndex
```

(3). 生成构建该子决策树的数据子集

```
def generateDataSet(dataSet, featLabels, n_features):  
    subDataSet = get_subsample(dataSet)  
    subFeature, subFeatIndex = get_subfeature(featLabels, n_features)  
    final_subData = []  
    for i in range(len(subDataSet)):  
        row_list = []  
        for j in range(n_features + 1):  
            row_list.append(subDataSet[i][subFeatIndex[j]])  
        final_subData.append(row_list)  
    return final_subData, subFeature
```

(4). 利用数据子集和 createTree 函数生成子决策树

```
myTree = createTree(final_subData, subFeature)
```

这样循环 n_trees 次构建了 n_trees 个子决策树，将所有子决策树都存放在 TreeList 列表中。

3. 根据随机森林计算每条测试集数据在每个子决策树上的分类情况

```
predList = []  
def cal(TreeList):  
    for j in range(len(test_set)):  
        classList1 = []  
        for i in range(len(TreeList)):  
            classLabel1 = classify(TreeList[i], featLabels, test_set[j])  
            classList1.append(classLabel1)  
        predList.append(classList1)  
cal(TreeList)
```

4. 投票决定每条数据的最终分类结果（调用了 Counter 函数用来记票数）

```
true_pred = []  
collection_words_list = []  
for i in range(len(test_set)):  
    collection_words_list.append(Counter(predList[i]))  
    true_pred.append(list(collection_words_list[i].keys())[0])
```

5. 计算准确率和 AUC 值

```
correct_num1 = 0  
for i in range(n):  
    if true_pred[i] == y_test[i]:  
        correct_num1 += 1  
rate1 = correct_num1 / n  
print('随机森林准确率为: ', rate1)  
  
auc_score1 = roc_auc_score(y_test, true_pred)  
print('随机森林AUC值:', auc_score1)
```

四、实验结果

决策树:

决策树准确率：0.8239043824701195
决策树训练集上准确率：0.8503083349910483
决策树AUC值：0.7444972877807385

随机森林：

随机森林准确率为：0.8262948207171315
随机森林AUC值：0.7294977636086791

1.算法对比分析：

维度	贪心决策树（ID3）	随机森林
优点	1. 模型直观，易于理解和解释 2. 可处理非线性关系，无需数据归一化 3. 可用于特征工程(通过信息增益筛选重要特征) 4. 训练速度较快	1. 通过集成降低过拟合风险，泛化能力更强 2. 对噪声和异常值更鲁棒 3. 能处理高维数据，特征随机性增加多样性 4. 袋外误差可近似验证集效果
缺点	1. 容易过拟合（尤其树深度大时） 2. 对数据微小变化敏感，树结构可能剧变 3. 偏向取值多的特征（ID3） 4. 无法很好处理缺失值	1. 模型解释性差（黑箱） 2. 训练和预测速度较慢（多棵树） 3. 需要调整参数(树的数量、特征子集大小) 4. 可能在小数据集上性能不如单棵决策树

2.在 Adult 数据集上的具体表现

根据实验运行结果：

模型	测试集准确率	训练集准确率	AUC 值
贪心决策树（ID3）	0.8239	0.8503	0.7445
随机森林（n_trees=10, n_features=8）	0.8263	（未提供）	0.7295

- **准确率：**随机森林略高于决策树（0.8263 > 0.8239），提升约 0.24 个百分点，优势不明显。
- **AUC 值：**决策树反而更高（0.7445 > 0.7295），说明决策树在正负类排序能力上略优。
- **过拟合程度：**决策树训练集准确率（0.8503）比测试集（0.8239）高约 2.6 个百分点，差距较小，说明在本数据集上决策树**未出现严重过拟合**。因此，随机森林“降低过拟合”的优势未能充分发挥。
- **结论：**在 Adult 数据集上，两种算法性能接近，随机森林未显著优于单棵决策树，甚至 AUC 指标略差。

3.针对“随机森林性能未显著优于决策树”的改进措施

决策树本身未过拟合，随机森林的集成优势无法体现，反而可能因参数设置不当或随机性引入噪声导致性能下降，以下是可能的解决方法：

A. 调整随机森林的超参数

- 增加树的数量（n_trees）：从 10 增加到 50、100 甚至 200。更多树可降低方差，提升稳定性。
- 调整特征子集大小（n_features）：原代码设为 8（总特征 13 左右）。可尝试更小（如 4~6）或更大

(如 10), 通过网格搜索找到最优。

- 限制树的最大深度: 当前每棵树完全生长(无剪枝)。可设置 `max_depth` (如 10~20), 防止单棵树过拟合噪声。

B. 改进特征选择与数据预处理

- 使用更好的分裂准则: ID3 的信息增益偏向多值特征, 可改用 C4.5 的信息增益率或 CART 的基尼指数。
- 处理类别不平衡: Adult 数据集中收入>50K 的样本比例较低(约 24%)。随机森林中可设置类别权重或采用过采样/欠采样方法。
- 特征工程: 当前离散化较粗糙(如年龄分 4 档), 可尝试更细的分箱或保留连续值使用阈值分裂。

C. 提升随机森林的多样性

- 增加特征随机性: 在每个节点分裂时, 随机选取的特征子集中加入更多噪声特征(但当前特征数较少, 可考虑构造交叉特征)。
- 使用不同的基学习器: 不仅限于 ID3, 可混用不同分裂准则的决策树。

五、实验体会

通过本次“决策树与随机森林分类”实验, 我对机器学习中的分类问题有了更深入的理解, 尤其在模型原理、数据预处理、编程实现和结果分析等方面收获颇丰。

首先, 在理论层面, 我掌握了决策树(ID3 算法)的核心思想——通过信息增益选择最优特征进行贪心分裂, 以及随机森林如何通过 Bootstrap 采样和随机特征子集构建多棵决策树并集成投票。理解了熵、条件熵、信息增益等数学公式的实际意义, 不再只是抽象概念。

其次, 数据预处理环节让我认识到真实数据集的复杂性。Adult 数据集包含缺失值、连续数值特征和类别特征, 需要逐一处理: 删除无关列(如 `fnlwgt`、`education`)、将连续值离散化(如年龄、工作时长)、合并稀疏类别(如国籍、职业)。这一过程锻炼了我对数据的观察和处理能力, 也体会到特征工程对模型性能的影响。

在编程实现上, 实验要求不直接调用 `scikit-learn` 的高层 API, 而是从零实现决策树和随机森林。这让我深入理解了递归构建树、计算信息增益、处理测试集未知分支等细节。特别是遇到测试集中出现训练集未曾见过的特征取值时, 我采用了简化的回退策略(按第一个存在分支处理), 虽然不够完美, 但让我意识到了决策树泛化能力的局限性以及实际工程中的权衡。

实验结果分析部分, 我对比了两种模型的准确率和 AUC 值。出乎意料的是, 随机森林在本数据集上并未显著优于单棵决策树, 甚至 AUC 略低。通过分析, 我认为主要原因是决策树本身过拟合不严重(训练集与测试集准确率仅差 2.6%), 因此随机森林降低过拟合的优势无法充分体现, 加上参数(树数量、特征子集大小)可能未达到最优, 以及数据离散化较粗糙, 导致集成效果不明显。这让我认识到: 集成学习并非在任何情况下都优于单模型, 需要根据数据特点调整策略。

针对上述问题, 我思考了改进措施, 如增加树的数量、调整特征子集、采用基尼指数或信息增益率、处理类别不平衡等。这些思考为今后优化模型性能提供了方向。

总之, 本次实验不仅巩固了机器学习的基础知识, 还提升了我的代码实现能力和问题分析能力。我深刻体会到理论与实践结合的重要性, 也明白了模型选择与调参需要依据具体数据灵活变通。这些经验将对我后续学习和研究机器学习算法产生积极帮助。

教师评语:

指导教师:

日 期: